

MAXIME BRONNY
19009314

FOUILLE DE DONNÉES

- DOCUMENTATION TECHNIQUE -

ARCHITECTURE, COMPOSANTS ET PIPELINE DE TRAITEMENT

PROJET DE CLUSTERING D'OFFRES D'EMPLOI PAR FOUILLE DE DONNÉES TEXTUELLES

PRÉSENTATION

Cette documentation technique présente un projet réalisé dans le cadre de l'UE Fouille de données textuelles du Master 1 Informatique, spécialisation Big Data, à l'Université Paris 8. Le travail porte sur la mise en œuvre d'un pipeline de clustering non supervisé appliqué à des offres d'emploi, afin de regrouper automatiquement des annonces selon leur contenu textuel.

L'objectif de ce document est de décrire l'organisation technique du projet, les principaux composants utilisés, les étapes du pipeline de traitement, ainsi que les choix méthodologiques retenus pour le prétraitement, la vectorisation, le clustering et l'évaluation. Il permet de comprendre la logique de fonctionnement du projet et la manière dont les résultats sont produits.

[HTTPS://GITHUB.COM/CRYZINX](https://github.com/cryzinx)

Table des matières

1	Présentation technique du projet	2
2	Organisation générale du dépôt	3
3	Description des composants principaux	4
3.1	Notebook principal	4
3.2	Module de prétraitement	4
3.3	Automatisation par le Makefile	5
4	Pipeline de traitement	5
4.1	Chargement et échantillonnage	5
4.2	Prétraitement textuel	5
4.3	Vectorisation TF-IDF	6
4.4	Clustering K-Means	6
4.5	Comparaison avec DBSCAN	7
4.6	Évaluation et visualisation	7
5	Dépendances et environnement	8
6	Entrées et sorties techniques	8
6.1	Données principales	8
6.2	Fichiers de validation externe	9
6.3	Sorties générées	9
7	Choix techniques principaux	9
8	Limites techniques et points de vigilance	10
9	Reproductibilité technique	11
9.1	Graine aléatoire	11
9.2	Dépendances fixées	11
9.3	Exécution non interactive	11
9.4	Ordre d'exécution recommandé	11
10	Conclusion	12

1. Présentation technique du projet

Ce projet implémente un pipeline de fouille de données textuelles dont l'objectif est de regrouper automatiquement des offres d'emploi LinkedIn en familles de métiers, à partir de leur contenu textuel et sans disposer d'étiquettes prédéfinies. L'ensemble du traitement est réalisé en Python dans un notebook Jupyter unique, accompagné d'un module de prétraitement dédié.

Le corpus source contient environ 124 000 offres d'emploi rédigées en anglais. Un échantillon de 30 000 offres est extrait aléatoirement pour le traitement. Les textes sont nettoyés par un module spécialisé qui retire notamment le boilerplate juridique récurrent dans les annonces, puis transformés en vecteurs numériques par TF-IDF et regroupés par l'algorithme K-Means avec $k = 8$ clusters.

Une comparaison avec l'algorithme DBSCAN est réalisée pour évaluer une approche alternative basée sur la densité. Les résultats sont évalués par des métriques internes (score silhouette) et par croisement avec des métadonnées structurées du dataset, notamment les secteurs d'activité et les catégories de compétences fournies par LinkedIn.

Le schéma suivant résume le flux global du pipeline :

```

postings.csv job_industries.csv
(124 000 offres) job_skills.csv
    | |
    v |
Echantillonnage |
(30 000 offres) |
    | |
    v |
Pretraitement |
(src/preprocessing.py) |
    | |
    v |
Vectorisation TF-IDF |
(30 000 x 10 000) |
    | |
    v |
Clustering K-Means |
(k = 8) |
    | |
    v v
Analyse des clusters <-- Validation externe
    |

```

```
v
Sorties (outputs/)
8 fichiers PNG
```

Le notebook constitue le point d'entrée unique. Le module `src/preprocessing.py` est importé par le notebook. Les fichiers de métadonnées (secteurs, compétences) interviennent uniquement en fin de pipeline pour la validation externe.

2. Organisation générale du dépôt

Le projet est organisé selon l'arborescence suivante.

```
.
+- Données/
| +- postings.csv # Dataset principal (~124 000 offres)
| +- compagnies/ # Métadonnées entreprises (non exploité)
| +- jobs/
| | +- job_industries.csv # Secteurs par offre
| | +- job_skills.csv # Compétences par offre
| +- mappings/
| +- industries.csv # Référentiel des secteurs (422 entrées)
| +- skills.csv # Référentiel des compétences (35 catégories)
+- notebooks/
| +- 01_clustering_offres_emploi_executed.ipynb
+- src/
| +- preprocessing.py # Module de nettoyage textuel
| +- __init__.py
+- scripts/
| +- generate_presentation_figures.py # Figures pour la presentation
+- outputs/ # Graphiques générés (8 fichiers PNG)
+- docs/ # cahier des charges, rapport, technique, utilisateur, presentation
+- requirements.txt
+- Makefile
+- README.md
```

Le dossier `Données/` contient le fichier CSV principal ainsi que les fichiers secondaires utilisés pour la validation externe. Le dossier `src/` regroupe le module Python de pré-traitement. Le dossier `outputs/` reçoit les graphiques produits automatiquement par le notebook. Le dossier `docs/` contient la documentation utilisateur et la documentation technique du projet.

3. Description des composants principaux

3.1 Notebook principal

Le fichier `notebooks/01_clustering_offres_emploi_executed.ipynb` constitue le point d'entrée du projet. Il orchestre l'ensemble du pipeline en six sections numérotées, chacune accompagnée de cellules Markdown qui expliquent les étapes et commentent les résultats.

La première section charge les données et effectue l'échantillonnage. La deuxième explore le corpus avec des statistiques descriptives et des visualisations. La troisième applique le pré-traitement textuel via le module `preprocessing.py`. La quatrième réalise la vectorisation TF-IDF. La cinquième exécute le clustering K-Means et la comparaison avec DBSCAN. La sixième analyse les clusters obtenus et les croise avec les métadonnées LinkedIn pour la validation externe.

Le notebook est conçu pour être exécuté séquentiellement, de la première à la dernière cellule. Les constantes configurables (`N_OFFRES`, `RANDOM_SEED`, `CHOSEN_K`) sont définies en début de notebook.

3.2 Module de prétraitement

Le fichier `src/preprocessing.py` (99 lignes) regroupe les fonctions de nettoyage textuel. Il expose quatre fonctions et deux structures de données.

La fonction `clean_html` supprime les balises HTML et décode les entités HTML courantes (`&`, `<`, etc.). La fonction `remove_boilerplate` retire les blocs de texte juridique et RH récurrents dans les offres d'emploi en s'appuyant sur la constante `BOILERPLATE_PATTERNS`, qui contient 15 expressions régulières compilées. Ces expressions ciblent les mentions d'employeur garantissant l'égalité des chances, les clauses relatives à la discrimination, les politiques de dépistage et d'autres formulations légales standardisées.

La fonction `clean_text` enchaîne le pipeline complet : suppression du HTML, passage en minuscules, retrait du boilerplate, suppression des caractères non alphabétiques, normalisation des espaces, tokenisation, filtrage des stopwords avec lemmatisation WordNet, et suppression des tokens de moins de trois caractères. Les stopwords comprennent les stopwords NLTK anglais auxquels s'ajoute l'ensemble `EEO_STOPWORDS`, un jeu de mots spécifiques au domaine EEO (*Equal Employment Opportunity*).

La fonction `build_corpus_text` construit le texte exploitable pour chaque offre en concaténant le titre (répété deux fois pour renforcer son poids sémantique) avec la description.

3.3 Automatisation par le Makefile

Le fichier `Makefile` à la racine du projet centralise les opérations courantes. Il définit les variables `PYTHON`, `PIP` et `JUPYTER` pointant vers les exécutables de l'environnement virtuel. Les cibles disponibles sont les suivantes.

Cible	Rôle
<code>make help</code>	Affiche la liste des cibles disponibles
<code>make venv</code>	Crée l'environnement virtuel Python
<code>make install</code>	Installe les dépendances (inclut <code>venv</code>)
<code>make notebook</code>	Lance Jupyter Notebook en mode interactif
<code>make run</code>	Exécute le notebook en non-interactif via <code>nbconvert</code>
<code>make docs</code>	Compile la documentation LaTeX (double passe)
<code>make clean</code>	Supprime les caches Python et fichiers temporaires LaTeX
<code>make clean-outputs</code>	Supprime les graphiques générés (<code>outputs/</code>)
<code>make tree</code>	Affiche l'arborescence utile du projet

La cible `make run` est la plus importante pour la reproductibilité : elle exécute le notebook de bout en bout via `jupyter nbconvert -execute`, sans intervention manuelle, et garantit que les résultats sont reproductibles. Les cibles `make notebook` et `make run` dépendent automatiquement de `make install`, ce qui assure que les dépendances sont toujours installées avant l'exécution.

4. Pipeline de traitement

Cette section décrit les étapes techniques du pipeline dans leur ordre d'exécution.

4.1 Chargement et échantillonnage

Le fichier `postings.csv` est chargé avec une sélection ciblée de colonnes (`title`, `description`, `formatted_experience_level`) pour limiter la consommation mémoire. Les lignes dont la description est absente sont supprimées. Un échantillon aléatoire de 30 000 offres est extrait avec la graine `RANDOM_SEED = 42` pour assurer la reproductibilité. Les descriptions de moins de 100 caractères sont ensuite filtrées car elles ne contiennent pas suffisamment de matière textuelle pour le clustering.

4.2 Prétraitement textuel

Le prétraitement est l'étape la plus déterminante pour la qualité du clustering et la plus longue en temps de calcul. Pour chaque offre, le titre est concaténé deux fois avec la

description par la fonction `build_corpus_text`, puis le texte résultant est nettoyé par `clean_text`.

Le nettoyage se déroule en huit étapes séquentielles : suppression des balises et entités HTML, passage en minuscules, retrait du boilerplate légal via les 15 expressions régulières, suppression des caractères non alphabétiques, normalisation des espaces, tokenisation par découpage sur les espaces, filtrage des stopwords (NLTK anglais et EEO) avec lemmatisation WordNet, et suppression des tokens de moins de trois caractères. Les documents nettoyés ont une longueur médiane d'environ 320 tokens.

Le retrait du boilerplate est une étape importante. Les mentions de type *equal opportunity employer* ou *affirmative action* sont présentes dans environ 30 % des descriptions. Sans ce nettoyage, ces termes récurrents dominent la représentation TF-IDF et produisent un cluster parasite qui regroupe les annonces par leur texte juridique plutôt que par leur contenu métier.

4.3 Vectorisation TF-IDF

Les textes nettoyés sont transformés en vecteurs numériques par le `TfidfVectorizer` de scikit-learn. Les paramètres retenus sont résumés dans le tableau suivant.

Paramètre	Valeur	Rôle
<code>max_features</code>	10 000	Taille maximale du vocabulaire
<code>max_df</code>	0.85	Exclut les termes dans plus de 85 % des documents
<code>min_df</code>	50	Exclut les termes dans moins de 50 documents
<code>sublinear_tf</code>	True	Pondération logarithmique ($1 + \log(\text{tf})$)
<code>ngram_range</code>	(1, 2)	Unigrammes et bigrammes

La matrice résultante est de dimension $30\,000 \times 10\,000$, stockée au format creux (*sparse*). Les bigrammes permettent de capturer des expressions composées discriminantes comme *registered nurse*, *data analyst* ou *customer service*. Le seuil `min_df` = 50 élimine les termes trop rares qui ajouteraient du bruit, et le seuil `max_df` = 0.85 écarte les termes trop fréquents qui ne sont pas discriminants.

4.4 Clustering K-Means

Le nombre de clusters est déterminé en testant les valeurs de k de 2 à 15. Pour chaque valeur, l'inertie (méthode du coude) et le score silhouette sont calculés. Le score silhouette est évalué sur un sous-échantillon de 10 000 documents pour limiter le temps de calcul.

La courbe d'inertie montre un ralentissement de la décroissance vers $k = 6$ à 8. Les scores silhouette restent faibles pour toutes les valeurs de k (inférieurs à 0.02), ce qui est

un comportement attendu en clustering textuel haute dimension. Le choix final repose sur l'interprétabilité qualitative : $k = 8$ produit des clusters thématiquement cohérents, tandis que des valeurs supérieures fragmentent les groupes en sous-catégories difficilement distinguables.

Le modèle K-Means final est entraîné avec $k = 8$, `n_init = 10` (dix initialisations aléatoires pour éviter les minima locaux) et `max_iter = 300`. Les huit clusters obtenus correspondent à des familles de métiers identifiées : emplois physiques, maintenance technique, commerce de détail, services généraux (résiduel), santé et soins infirmiers, informatique et ingénierie, direction commerciale et management.

4.5 Comparaison avec DBSCAN

DBSCAN est testé comme approche alternative. Contrairement à K-Means, cet algorithme ne nécessite pas de spécifier le nombre de clusters à l'avance et peut identifier des points de bruit, ce qui est potentiellement intéressant pour les offres atypiques.

Avant d'appliquer DBSCAN, les données TF-IDF sont réduites à 50 dimensions par TruncatedSVD, car DBSCAN est sensible à la haute dimension. Quatre valeurs du paramètre epsilon sont testées (0.5, 1.0, 1.5, 2.0) avec `min_samples = 10`.

Les résultats montrent que DBSCAN ne produit pas de regroupements exploitables sur ces données. Pour les valeurs d'epsilon faibles, la majorité des documents sont classés comme bruit. Pour les valeurs élevées, un cluster unique absorbe la quasi-totalité des documents. Ce comportement est caractéristique des données textuelles vectorisées, où les distances entre documents sont relativement uniformes en haute dimension. K-Means est donc retenu comme méthode principale.

4.6 Évaluation et visualisation

L'évaluation des clusters repose sur trois axes complémentaires.

Le premier est l'analyse interne par le score silhouette. Le score global et les scores par cluster sont calculés, ainsi que le pourcentage de documents ayant un score positif dans chaque cluster. Sept clusters sur huit présentent un pourcentage de scores positifs supérieur à 80 %, ce qui confirme leur cohérence interne malgré un score global faible.

Le deuxième est l'interprétation qualitative. Les dix termes les plus représentatifs de chaque cluster sont extraits depuis les centroïdes du modèle K-Means. Les trois offres les plus proches de chaque centroïde sont affichées pour vérifier la cohérence des regroupements. Une projection en deux dimensions par TruncatedSVD (`n_components = 2`) permet de visualiser la répartition spatiale des clusters.

Le troisième est la validation externe par croisement avec les métadonnées LinkedIn. La distribution des niveaux d'expérience est analysée par cluster. Les fichiers `job_industries.csv` et `job_skills.csv`, associés à leurs tables de référence, permettent de croiser les clusters textuels avec les secteurs d'activité et les catégories de compétences déclarées sur LinkedIn. Ce croisement vérifie que les regroupements obtenus à partir du texte correspondent à des catégories métier réelles.

5. Dépendances et environnement

Le projet a été développé avec Python 3.12 sur un système Linux. Il est compatible avec les systèmes courants (Linux, macOS, Windows) à condition de disposer de Python 3.10 ou supérieur. Les dépendances sont spécifiées dans le fichier `requirements.txt`.

- `pandas` (≥ 2.0) pour la manipulation des données tabulaires
- `numpy` (≥ 1.24) pour le calcul numérique
- `scikit-learn` (≥ 1.3) pour la vectorisation TF-IDF, le clustering K-Means, DB-SCAN, TruncatedSVD et les métriques d'évaluation
- `nltk` (≥ 3.8) pour les stopwords anglais et la lemmatisation WordNet
- `matplotlib` (≥ 3.7) et `seaborn` (≥ 0.12) pour les visualisations
- `jupyter` (≥ 1.0) pour l'environnement de notebook

Le module `preprocessing.py` télécharge automatiquement les ressources NLTK nécessaires (`stopwords` et `wordnet`) lors de son importation. Il est recommandé de disposer d'au moins 8 Go de mémoire vive pour exécuter le pipeline sans difficulté.

6. Entrées et sorties techniques

6.1 Données principales

Le fichier principal est `Données/postings.csv`, issu du dataset *LinkedIn Job Postings (2023)* disponible sur Kaggle¹. Il pèse environ 500 Mo et contient 124 000 offres d'emploi LinkedIn en anglais. Les deux champs exploités pour le clustering sont `title` (titre du poste, toujours renseigné, environ 31 caractères en moyenne) et `description` (description complète, toujours renseignée, environ 3 750 caractères en moyenne). Le champ `formatted_experience_level` est utilisé uniquement pour la validation croisée après clustering, avec une couverture de 76 %.

¹<https://www.kaggle.com/datasets/arshkon/linkedin-job-postings>

6.2 Fichiers de validation externe

Quatre fichiers secondaires servent à la validation des clusters. Le fichier `jobs/job_industries.csv` associe chaque offre à un identifiant de secteur d'activité (couverture de 98.8%). Le fichier `mappings/industries.csv` fournit les noms des 422 secteurs. Le fichier `jobs/job_skills.csv` associe chaque offre à une catégorie de compétence (couverture de 98.6%). Le fichier `mappings/skills.csv` fournit les noms des 35 catégories.

Les fichiers relatifs aux entreprises, aux salaires et aux avantages sociaux ne sont pas exploités. Les fichiers entreprises ne sont pas pertinents car le clustering porte sur le contenu des offres. Les fichiers salaires (couverture de 29%) et avantages (couverture de 23%) ont été écartés en raison de leur couverture insuffisante.

6.3 Sorties générées

Le notebook produit huit fichiers PNG dans le dossier `outputs/`.

- `distribution_longueurs.png` : distribution des longueurs des titres et des descriptions
- `distribution_tokens.png` : distribution du nombre de tokens après nettoyage
- `elbow_silhouette.png` : courbe du coude et scores silhouette pour $k = 2$ à 15
- `taille_clusters.png` : taille de chaque cluster
- `clusters_2d.png` : projection 2D des clusters par TruncatedSVD
- `experience_par_cluster.png` : niveaux d'expérience par cluster
- `industries_par_cluster.png` : croisement clusters et secteurs d'activité
- `skills_par_cluster.png` : croisement clusters et catégories de compétences

Le notebook affiche également dans ses cellules de sortie les termes dominants par cluster, des exemples d'offres représentatives et les métriques d'évaluation détaillées.

7. Choix techniques principaux

La représentation par TF-IDF a été retenue pour sa simplicité et son interprétabilité. Contrairement aux plongements de mots (Word2Vec, BERT), TF-IDF produit des vecteurs directement liés au vocabulaire du corpus, ce qui facilite l'analyse des clusters par extraction des termes dominants. Cette transparence est un avantage important dans un contexte de projet universitaire où la compréhension du résultat prime sur la performance brute.

K-Means a été choisi après comparaison avec DBSCAN. L'algorithme K-Means impose un nombre fixe de clusters, ce qui permet de contrôler la granularité du regroupement et de comparer différentes valeurs de k de manière systématique. DBSCAN, testé avec réduction dimensionnelle par TruncatedSVD, s'est révélé inadapté aux données textuelles creuses en haute dimension, où les distances entre documents sont relativement uniformes.

Le retrait du boilerplate légal par expressions régulières constitue un choix pragmatique. Les offres d'emploi contiennent systématiquement des blocs juridiques standardisés (clauses relatives à la discrimination, politique d'égalité des chances) qui, sans nettoyage, dominent la représentation vectorielle et forment un cluster parasite. Les 15 expressions régulières couvrent les cas les plus fréquents du corpus anglophone.

La lemmatisation WordNet a été préférée au stemming (Porter, Snowball) car elle produit des formes de mots valides, ce qui rend les termes dominants des clusters directement lisibles et interprétables. Le coût supplémentaire en temps de calcul reste acceptable sur 30 000 documents.

La validation externe par croisement avec les métadonnées LinkedIn (secteurs d'activité et compétences) a été ajoutée pour compenser la faiblesse du score silhouette. Cette approche permet de vérifier que les clusters textuels correspondent à des catégories métier réelles, indépendamment de la métrique interne.

8. Limites techniques et points de vigilance

La représentation TF-IDF ne capture pas les relations sémantiques entre les mots. Deux offres portant sur le même métier mais rédigées avec un vocabulaire différent peuvent se retrouver dans des clusters distincts. Des approches par plongements contextuels (BERT, Sentence-BERT) pourraient améliorer cet aspect, au prix d'une complexité et d'un temps de calcul nettement supérieurs.

Le score silhouette global est faible (environ 0.01). Ce résultat est attendu en clustering textuel haute dimension, car cette métrique est conçue pour des clusters compacts en basse dimension et ne reflète pas correctement la qualité des regroupements dans un espace creux à 10 000 dimensions. L'évaluation repose donc principalement sur l'interprétabilité qualitative et la validation externe.

Le cluster 3 (services généraux) fonctionne comme un groupe résiduel qui rassemble les offres ne s'intégrant pas clairement dans les autres catégories. C'est un comportement classique de K-Means, qui assigne chaque document à un cluster sans possibilité de rejet. Malgré un nettoyage renforcé, certains patterns de boilerplate peuvent subsister et influencer marginalement ce cluster.

Le corpus est entièrement en anglais. Les stopwords, le lemmatiseur et les expressions régulières de boilerplate sont spécifiques à cette langue. Le pipeline n'est pas directement transposable à des offres en français sans adaptation du module de prétraitement.

L'exécution du pipeline nécessite environ 8 Go de mémoire vive et 500 Mo d'espace disque pour le fichier de données principal. Le temps d'exécution complet est d'environ 5 minutes sur une machine standard.

9. Reproductibilité technique

La reproductibilité des résultats est assurée par plusieurs mécanismes.

9.1 Graine aléatoire

La constante `RANDOM_SEED = 42`, définie en début de notebook, contrôle l'échantillonnage des données et l'initialisation de K-Means. Avec la même graine et les mêmes données en entrée, le pipeline produit des résultats identiques.

9.2 Dépendances fixées

Le fichier `requirements.txt` spécifie les versions minimales des bibliothèques. L'utilisation d'un environnement virtuel (`venv/`) isole les dépendances du projet et évite les conflits avec d'autres installations Python.

9.3 Exécution non interactive

La commande `make run` exécute le notebook de bout en bout via `jupyter nbconvert -execute`, sans intervention manuelle. Cette cible garantit que l'ensemble du pipeline est exécuté dans l'ordre, de la première à la dernière cellule, et que les résultats sont reproductibles indépendamment de l'état précédent du notebook.

9.4 Ordre d'exécution recommandé

Pour reproduire les résultats depuis un état initial :

1. `make install` : crée l'environnement virtuel et installe les dépendances
2. Placer les fichiers du dataset dans `Données/`
3. `make run` : exécute le notebook et génère les sorties
4. `make docs` : compile la documentation (optionnel)

La commande `make clean-outputs` permet de supprimer les sorties existantes avant une ré-exécution propre.

10. Conclusion

Cette documentation décrit l'architecture technique et le fonctionnement du pipeline de clustering d'offres d'emploi. Le projet repose sur une chaîne de traitement classique en fouille de données textuelles (prétraitement, TF-IDF, K-Means) implémentée dans un notebook Jupyter unique, accompagné d'un module de prétraitement dédié et d'un Makefile pour l'automatisation.

Les choix techniques sont guidés par la simplicité, l'interprétabilité et la reproductibilité. Malgré les limites inhérentes à TF-IDF et K-Means, les clusters obtenus correspondent à des familles de métiers cohérentes, ce que confirme le croisement avec les métadonnées structurées du dataset LinkedIn. Le pipeline reste ouvert à des améliorations, notamment l'utilisation de représentations vectorielles denses ou de méthodes de clustering hiérarchique.